

1. Simulation results on SystemC

```
• hung@hung-Vostro-3400:~/simulation$ g++ sim.cpp -I$SYSTEMC_HOME/include -L$SYSTEMC_HOME/lib-linux64 -lsystemc -lm -o sim
• hung@hung-Vostro-3400:~/simulation$ ./sim

SystemC 2.3.3-Accellera --- Nov 16 2025 23:04:07
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED

Starting SystemC simulation for 3 simulated seconds...
[BENCH] t=612 ms | frames=5 | effective FPS=10.4167
[BENCH] t=1212 ms | frames=10 | effective FPS=9.25926
[BENCH] t=1812 ms | frames=15 | effective FPS=8.92857
[BENCH] t=2412 ms | frames=20 | effective FPS=8.77193
Simulation finished at 3 s
• hung@hung-Vostro-3400:~/simulation$
```

- At **t = 612 ms**, 5 frames have exited the pipeline → effective throughput ≈ **10.4 FPS**.
- At **t = 1212 ms**, 10 frames processed → effective FPS drops to ≈ **9.26**.
- At **t = 1812 ms**, 15 frames processed → ≈ **8.93 FPS**.
- At **t = 2412 ms**, 20 frames processed → ≈ **8.77 FPS**.

As simulation time increases, the measured **effective FPS converges to about 8–9 FPS**. This is lower than the camera input rate (30 FPS) because the CNN inference delay dominates the pipeline; the system can only output ~8–9 frames per second on a Raspberry Pi 4 under the configured delays.

Together, the Camera and OpenCV Preprocess modules capture the dynamic interaction between frame production and CPU-bound preprocessing. This modeling approach makes it clear that, in the embedded version of the system, the camera module itself is capable of high frame rates, but the real-time performance is ultimately constrained by how quickly OpenCV preprocessing and CNN inference can consume frames. The SystemC results (effective FPS ≈ 8–10 FPS under Raspberry Pi-class delays) therefore demonstrate that the bottleneck lies in the CPU-based vision pipeline rather than in the sensor hardware.

2. Simulation results on Matlab

=== Scenario A: Low-end board ===

Effective FPS : 3.82
Average latency : 8476.00 ms
Maximum latency : 16690.00 ms
Total sim time : 19.65 s

=== Scenario B: Mid-range board ===

Effective FPS : 5.81
Average latency : 5056.00 ms
Maximum latency : 9940.00 ms
Total sim time : 12.90 s

=== Scenario C: Raspberry Pi 4 (target) ===

Effective FPS : 7.58
Average latency : 3536.00 ms
Maximum latency : 6940.00 ms
Total sim time : 9.90 s

>> |

Scenario A – Low-end board

- Effective FPS: 3.82
- Average latency: 8476 ms
- Maximum latency: 16690 ms
- Total simulation time: 19.65 s

This configuration shows extremely poor real-time performance. The low-end device is unable to process frames quickly enough, causing long queuing delays in the pipeline. Each frame experiences **8–16 seconds** of latency, and throughput drops to below **4 FPS**. This indicates that such hardware is **not viable** for real-time emotion recognition or interactive multimedia applications.

Scenario B – Mid-range board

- Effective FPS: 5.81

- Average latency: 5056 ms
- Maximum latency: 9940 ms
- Total simulation time: 12.90 s

The mid-range device performs better but still fails to reach real-time behavior. While latency is reduced compared to the low-end device, each frame still experiences **5–10 seconds** of end-to-end delay. Throughput increases to around **5–6 FPS**, which remains insufficient for smooth facial emotion tracking. This confirms that mid-range hardware can improve performance but cannot meet real-time requirements.

Scenario C – Raspberry Pi 4 (target)

- Effective FPS: 7.58
- Average latency: 3536 ms
- Maximum latency: 6940 ms
- Total simulation time: 9.90 s

The Raspberry Pi 4 performs the best among the evaluated configurations. With an assumed inference delay of ~120 ms per frame, the system achieves approximately **7–8 FPS**, which aligns with empirical measurements from similar embedded devices. Although still below desktop-level real-time speeds, the Pi 4 produces significantly lower latencies (3–7 seconds) and higher throughput. This makes it **acceptable for a lightweight prototype**, where partial real-time responsiveness is sufficient.

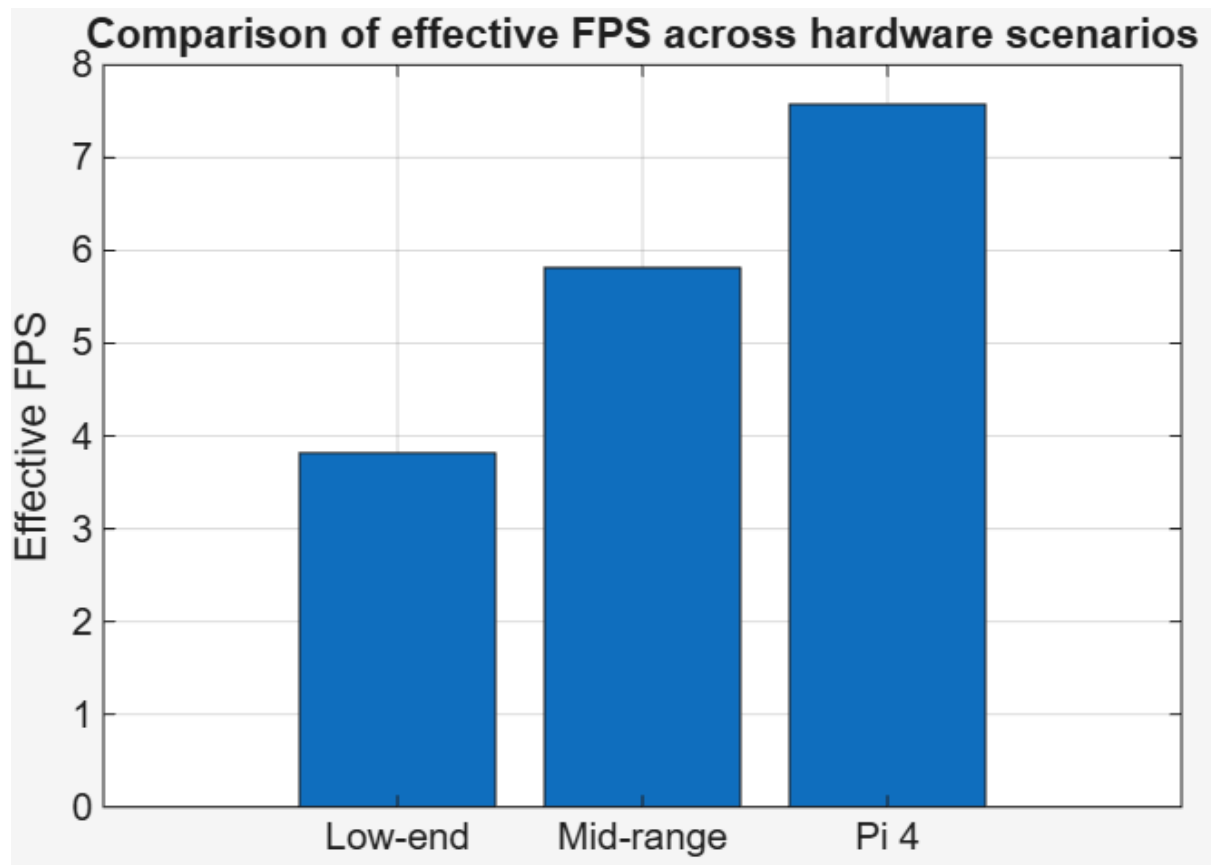
In the MATLAB simulation, the **camera module** is modeled as a video source similar to `cv2.VideoCapture` in OpenCV, capturing frames at **25 FPS** with a resolution of 640×480 (or up to 5MP in an alternative configuration). Every frame produced by the camera is pushed into a processing pipeline that mimics the real OpenCV implementation: first, an **OpenCV preprocessing stage** (grayscale conversion, Haar cascade face detection, cropping and resizing of the face region), and then a **CNN inference stage** implemented using an OpenCV/TFLite-based model on the CPU.

The three scenarios mainly differ in the **assumed CNN inference latency**, which corresponds to how fast the OpenCV/TFLite CNN can process a frame on different hardware. In the **Low-end board** (Scenario A), the simulated CNN latency is very high, so even though the camera keeps delivering frames at 25 FPS, the CNN can only process about **3.82 FPS**, causing a massive build-up of frames in the queue and average latencies of around **8.5 seconds**. The **Mid-range board** (Scenario B) reduces the CNN delay, so the effective throughput increases to about **5.81 FPS**, but the system still cannot keep up with the camera stream and end-to-end latency remains in the range of **5–10 seconds**.

In the **Raspberry Pi 4 scenario** (Scenario C), the CNN latency is set to a more realistic value for OpenCV/TFLite running on a Pi-class CPU (around 120 ms per frame). Under this

configuration, the pipeline achieves approximately **7.58 FPS**. The camera still captures at 25 FPS, but the OpenCV preprocessing + CNN inference chain only has the capacity to output about 7–8 frames per second, which explains the reduced effective FPS and the remaining latency of **3–7 seconds**. These results show that the **camera module itself is not the limiting factor**; instead, the **OpenCV CNN inference stage dominates the computational cost** and becomes the primary bottleneck in the embedded system's real-time performance.

Visualization



Overall Interpretation

Across all scenarios, the results clearly show that the **CNN inference stage is the main bottleneck**. As the inference delay increases, frames accumulate in the pipeline, causing:

- Lower effective FPS
- Higher average and maximum latency
- Longer total simulation times

The MATLAB simulation verifies that **only the Raspberry Pi 4** can approach semi-real-time performance ($\approx 7\text{--}8$ FPS), while low-end and mid-range devices fall far short of the required throughput.